Project Proposal: Empirical and Theoretical Analysis of the Adam Optimization Algorithm

ECMC1018: Analysis of Algorithms (Análisis de Algoritmos)

Dario Pomasqui
School of Mathematical and Computational Sciences
Yachay Tech University, San Miguel de Urcuquí, Ecuador

Course: ECMC1018 – Analysis of Algorithms | **Instructor:** Prof. Israel Gustavo Pineda Arias

Date: September 11, 2026

Abstract

Stochastic optimization is a cornerstone of modern machine learning, enabling the efficient training of high-dimensional differentiable models. Among first-order stochastic methods, Adam (Adaptive Moment Estimation) has emerged as one of the most widely adopted algorithms due to its fast initial convergence and parameter-wise adaptive step sizes. However, theoretical counterexamples highlight scenarios where Adam fails to converge to an optimal solution, raising important questions about its stability and hyperparameter sensitivity. Building upon our practical experience in machine learning model development, this project proposal presents a rigorous empirical and theoretical investigation of Adam. We outline a systematic framework to analyze Adam's asymptotic complexity, evaluate its empirical performance against Stochastic Gradient Descent (SGD) with momentum, RMSProp, and AMSGrad across deep neural network architectures, and analyze its behavior on non-convex loss landscapes and theoretical counterexample functions.

Keywords: Adam Optimizer, Stochastic Gradient Descent, Non-Convex Optimization, Algorithmic Complexity, Machine Learning, AMSGrad.

1 Introduction and Problem Statement

Stochastic optimization lies at the core of training large-scale differentiable models in machine learning. First-order optimization algorithms compute parameter updates using noisy gradient estimates obtained from subset mini-batches of data. While standard Stochastic Gradient Descent (SGD) applies a uniform learning rate across all parameter dimensions, complex loss landscapes—characterized by ill-conditioned curvature, sparse gradients, or disparate feature scales—often cause SGD to exhibit slow convergence or severe oscillations.

To address these challenges, Kingma and Ba (2015) introduced **Adam** (Adaptive Moment Estimation). Adam combines the principles of momentum (tracking the exponentially decaying average of past gradients) and RMSProp (scaling updates by the square root of past squared gradients).

Algorithmic Formulation of Adam

Let $J(\theta)$ denote a differentiable objective function parameterized by $\theta \in \mathbb{R}^d$. At iteration t, given a stochastic gradient $g_t = \nabla_\theta J_t(\theta_t)$, hyperparameter learning rate $\alpha > 0$, exponential decay rates $\beta_1, \beta_2 \in [0, 1)$, and smoothing parameter $\varepsilon > 0$, the algorithm updates state vectors according to:

1. **First Moment Update (Momentum):**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{1}$$

2. **Second Uncentered Moment Update (Adaptive Scaling):**

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{2}$$

   (where g_t^2 represents coordinate-wise squaring).

3. **Initialization Bias Correction:** Since $m_0 = 0$ and $v_0 = 0$, both vectors are biased towards zero in early iterations. The bias-corrected estimators are computed as:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{3}$$

4. **Parameter Update Rule:**

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{4}$$

Algorithm 1: Adam Optimization Algorithm

Require: Step size α, decay rates $\beta_1, \beta_2 \in [0, 1)$, objective $J(\theta)$, initial parameters $\theta_0 \in \mathbb{R}^d$, smoothing constant $\varepsilon = 10^{-8}$.

Initialize: $m_0 \leftarrow 0$ (1st moment vector), $v_0 \leftarrow 0$ (2nd moment vector), $t \leftarrow 0$.

1. **while** θ_t has not converged **do**
2. $\quad t \leftarrow t + 1$
3. $\quad g_t \leftarrow \nabla_\theta J_t(\theta_{t-1})$ *// Compute stochastic gradient*
4. $\quad m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ *// Update 1st moment estimate*
5. $\quad v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ *// Update 2nd raw moment estimate*
6. $\quad \hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ *// Compute bias-corrected 1st moment*
7. $\quad \hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ *// Compute bias-corrected 2nd moment*
8. $\quad \theta_t \leftarrow \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$ *// Update parameter vector*
9. **end while**
10. **return** θ_t

Despite Adam's widespread empirical popularity, theoretical investigations (e.g., Reddi et al., 2018) proved that Adam's effective step-size ratio can increase non-monotonically, leading to non-convergence even on simple convex optimization problems. This project addresses the fundamental tension between Adam's practical efficiency and its theoretical limits.

2 Core Research Questions

To conduct a structured analysis of Adam within the scope of algorithmic performance and machine learning optimization, this project answers six core research questions:

2.1 1. What kinds of problems can it solve?

Adam optimizes large-scale, high-dimensional stochastic objective functions. It is designed for training differentiable supervised learning models, such as Multilayer Perceptrons (MLPs), Convolutional Neural Networks (CNNs), and deep regression architectures. Having previously developed and trained machine learning models, we leverage Adam's ability to automatically adjust coordinate-wise step sizes across complex model parameters. While it operates effectively on non-convex landscapes in practice, global optimality guarantees do not generally hold.

2.2 2. What are its theoretical and operational assumptions?

At the operational level, Adam assumes that a first-order stochastic gradient g_t can be computed efficiently per mini-batch, the loss function is differentiable with respect to θ, and valid hyperparameters are chosen ($\alpha > 0$, $\varepsilon > 0$, $0 \leq \beta_1, \beta_2 < 1$). Theoretically, early convergence proofs assumed bounded gradients and monotonic step sizes; however, non-monotonicity in $\hat{v}_t$ violates these assumptions in specific convex settings.

2.3 3. What constitutes a rigorous empirical benchmark?

An effective empirical benchmark compares Adam against SGD with Momentum, RMSProp, and AMSGrad on standard classification datasets (MNIST and Fashion-MNIST) using fixed neural network topologies (`MLPClassifier` and PyTorch models). The evaluation metrics include training loss trajectories, validation accuracy, convergence rate (number of iterations to reach a target loss ϵ), and sensitivity to batch sizes and learning rates.

2.4 4. What data representation is appropriate?

Input data is represented as a real-valued matrix $X \in \mathbb{R}^{n \times d}$, where n is the number of samples and d is the feature dimension, along with target labels $y \in \mathbb{R}^n$. Mini-batches $X_B \subset X$ are sampled stochastically to evaluate loss gradients $\nabla_\theta J_B(\theta)$. All continuous attributes are standardized to zero mean and unit variance. The optimizer operates strictly on floating-point parameter tensors $\theta \in \mathbb{R}^d$ and gradient vectors $g \in \mathbb{R}^d$.

2.5 5. What competing approaches should be evaluated?

The primary baseline is **Stochastic Gradient Descent (SGD) with Momentum**, representing the standard first-order method without parameter-wise step-size adaptation. Secondary baselines include **RMSProp** (to isolate the impact of momentum) and **AMSGrad** (which guarantees monotonic step sizes $\hat{v}_t \geq \hat{v}_{t-1}$ to fix Adam's non-convergence bug). Fair comparisons require matching hyperparameter search budgets.

2.6 6. What specific behaviors are worth investigating?

We will investigate three critical phenomena:

(a) *Convergence Speed vs. Generalization Gap:* Evaluating whether Adam converges faster in training loss but generalizes worse on test sets compared to SGD with Momentum.

(b) *Hyperparameter Sensitivity:* Analyzing performance degradation under non-optimal choices of α, β_1, and β_2.

(c) *Non-Convergence Reproduction:* Implementing Reddi et al.'s 1D synthetic convex counterexample function to empirically demonstrate Adam's divergence and verify AMSGrad's corrective behavior.

3 Methodology and Implementation Strategy

The methodology is organized into three complementary tracks combining theoretical complexity analysis, software implementation, and empirical benchmarking.

3.1 Theoretical & Algorithmic Complexity Analysis

We evaluate Adam's efficiency relative to standard first-order solvers:

Table 1: Asymptotic Complexity & Memory Requirements per Step

Algorithm	**Time Comp.**	**Aux. Memory**	**State Saved**	**Step Adaptation**
SGD	$\mathcal{O}(d)$	$\mathcal{O}(1)$	None	Uniform
SGD + Momentum	$\mathcal{O}(d)$	$\mathcal{O}(d)$	m_t	Uniform
RMSProp	$\mathcal{O}(d)$	$\mathcal{O}(d)$	v_t	Parameter-wise
Adam	$\mathcal{O}(d)$	$\mathbf{2\mathcal{O}(d)}$	m_t, v_t	Parameter-wise
AMSGrad	$\mathcal{O}(d)$	$3\mathcal{O}(d)$	$m_t, v_t, \hat{v}_t^{\max}$	Monotonic

Note: For parameter dimension d, Adam requires $2d$ additional floating-point storage values for m_t and v_t. The per-iteration computational overhead is $8d$ floating-point operations (additions, multiplications, divisions, and square roots), maintaining an asymptotic time complexity of $\mathcal{O}(d)$ per step.

3.2 Empirical Benchmarking Framework

All experiments will be implemented in Python using PyTorch and Scikit-learn, structured into a clean modular evaluation repository:

- **Synthetic Optimization Objectives:** 2D Rastrigin, Rosenbrock, and Reddi's 1D non-convergence function.
- **Real-World Datasets:** MNIST (28×28 grayscale digits, $n = 60,000$) and Fashion-MNIST ($n = 60,000$).
- **Model Architectures:** 3-layer Multilayer Perceptron (MLP) with ReLU activations and Softmax output.

3.3 Hyperparameter Grid Search Strategy

To ensure unbiased optimizer evaluation, we follow Choi et al. (2019) by conducting grid searches over parameter grids:

$$\begin{aligned} &\alpha \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}, && \beta_1 \in \{0.8, 0.9, 0.99\}, \\ &\beta_2 \in \{0.99, 0.999\}, && B \in \{32, 64, 128, 256\} \end{aligned} \tag{5}$$

4 Project Work Plan and Milestone Schedule

The project schedule is aligned with the course calendar for **ECMC1018 Analysis of Algorithms**, spanning from **September 11, 2026** through the major project presentations and final evaluation.

Table 2: Project Work Plan and Key Course Milestones

Phase	Timeline	Key Tasks & Deliverables	Course Milestone
Phase 1	Weeks 5–7 (Sep 11 – Oct 02)	Mathematical foundation review, setup of Python experimental codebase, implementation of synthetic benchmark functions and Reddi counterexample.	Initial Repository Setup & Synthetic Tests
Phase 2	Weeks 7–9 (Oct 02 – Oct 16)	PyTorch solver implementation for Adam, SGD+Momentum, RMSProp, and AMSGrad; data preprocessing pipelines on MNIST & Fashion-MNIST.	Modular Solver Implementation
Phase 3	Weeks 9–11 (Oct 16 – Oct 23)	Systematic hyperparameter grid search execution, convergence trajectory logging, and slide deck preparation.	**Project Presentation I (Oct 23, 2026)**
Phase 4	Weeks 12–15 (Oct 24 – Nov 17)	Loss landscape visualization, generalization gap analysis, final empirical synthesis, and technical report writing.	**Project Presentation II (Nov 17–20, 2026)**
Phase 5	Weeks 16–17 (Nov 21 – Dec 01)	Code freeze, reproducible artifact packaging, final examination preparation, and course grade review.	**Final Exam (Nov 27)** **Last Day (Dec 01)**

5 Expected Contribution

This project bridges theoretical algorithm analysis with practical machine learning optimization. The principal contributions of this work include:

1. **An Open-Source Reproducible Benchmark Repository:** A modular Python/PyTorch codebase comparing first-order stochastic optimizers under uniform tuning conditions.
2. **Empirical Validation of Theoretical Counterexamples:** Quantitative verification of Adam's divergence on non-monotonic step-size landscapes (Reddi et al., 2018) and demonstration of AMSGrad's corrective mechanics.
3. **Practical Selection Guidelines for Practitioners:** Evidence-based recommendations detailing when Adam outperforms SGD+Momentum, and when its adaptive scaling introduces generalization penalties.

References

[1] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015. arXiv:1412.6980.

[2] D. Choi, C. Shallue, Z. Nado, J. Lee, C. J. Maddison, and G. E. Dahl, "On Empirical Comparisons of Optimizers for Deep Learning," *arXiv preprint arXiv:1910.05446*, 2019. arXiv:1910.05446.

[3] S. J. Reddi, S. Kale, and S. Kumar, "On the Convergence of Adam and Beyond," in *International Conference on Learning Representations (ICLR)*, Vancouver, Canada, 2018. OpenReview.

[4] Scikit-learn developers, "MLPClassifier: Multilayer Perceptron Classifier Documentation," version 1.9.0, 2026. Scikit-Learn Docs.